

Avaliação de Afinidades de Threads no Solver Navier-Stokes

I. Introdução

A forma como as threads são mapeadas aos núcleos do processador pode impactar significativamente o desempenho de aplicações paralelas. Nesta tarefa, avaliamos como a escalabilidade do solver Navier-Stokes se comporta sob diferentes configurações de afinidade de threads suportadas pelo OpenMP e pelo sistema operacional no mesmo nó de computação do NPAD.

Como o uso de `OMP_PROC_BIND=false` delega o controle de afinidade totalmente ao sistema operacional, invalidando as diretivas de `OMP_PLACES`, os testes foram focados em três cenários distintos para uma comparação justa:

- **Baseline (Sem afinidade):** `OMP_PROC_BIND=false`
- **Afinidade por Núcleo Físico:** `OMP_PLACES=cores` com `OMP_PROC_BIND=true`
- **Afinidade por Thread Lógica (SMT):** `OMP_PLACES=threads` com `OMP_PROC_BIND=true`

II. Metodologia

Os testes foram executados no nó [amd-512](#) do NPAD, que possui 256 núcleos (512 threads lógicas com SMT). O código utilizado foi o solver Navier-Stokes, paralelizado com OpenMP.

Para cada configuração, foram variados:

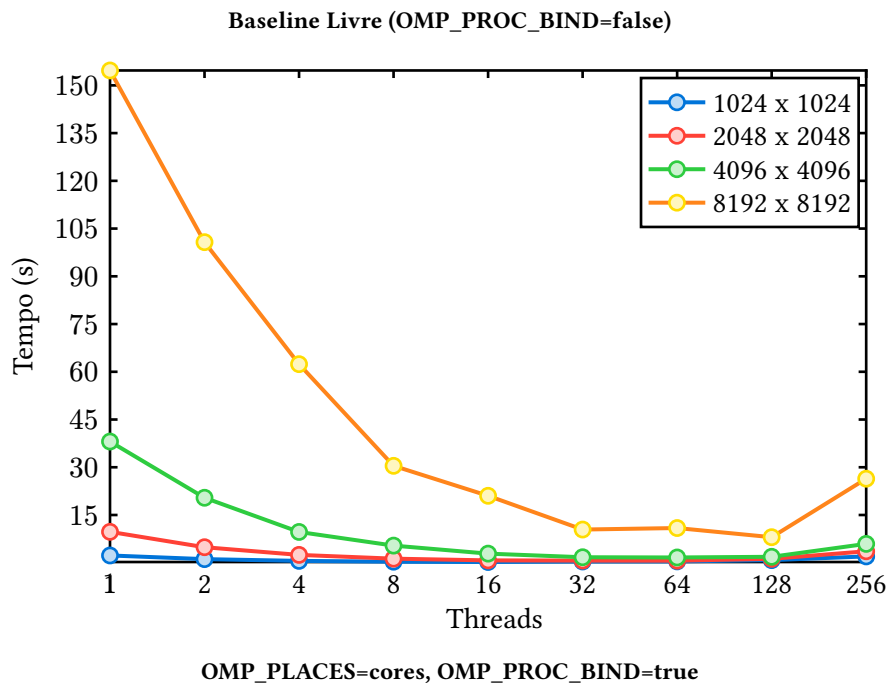
- **Tamanho do grid:** 1024×1024, 2048×2048, 4096×4096 e 8192×8192
- **Quantidade de threads:** 1, 2, 4, 8, 16, 32, 64, 128 e 256
- **Passos temporais:** 5000 para todas as execuções

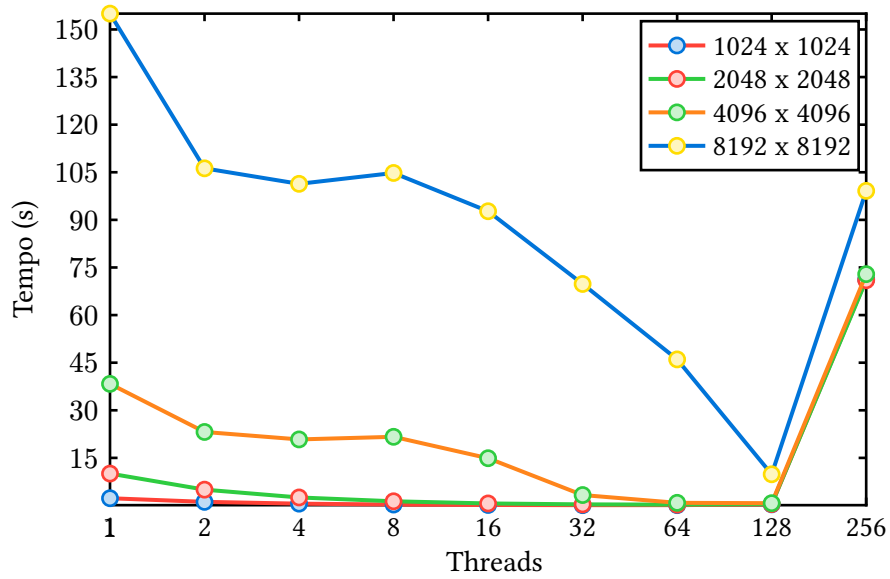
Ao todo, cada configuração gerou 36 execuções (4 grids × 9 contagens de threads). Para a representação da baseline (BIND=**false**), utilizou-se o conjunto de dados gerado pela execução padrão livre.

III. Resultados

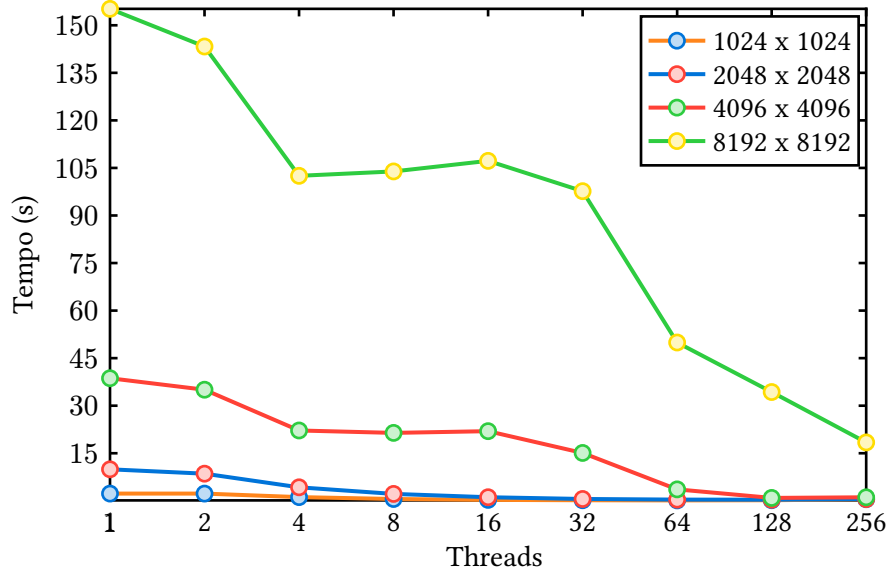
III. I. Desempenho por Configuração de Afinidade

Abaixo são apresentados os gráficos de tempo de execução em função do número de threads para cada um dos três cenários.



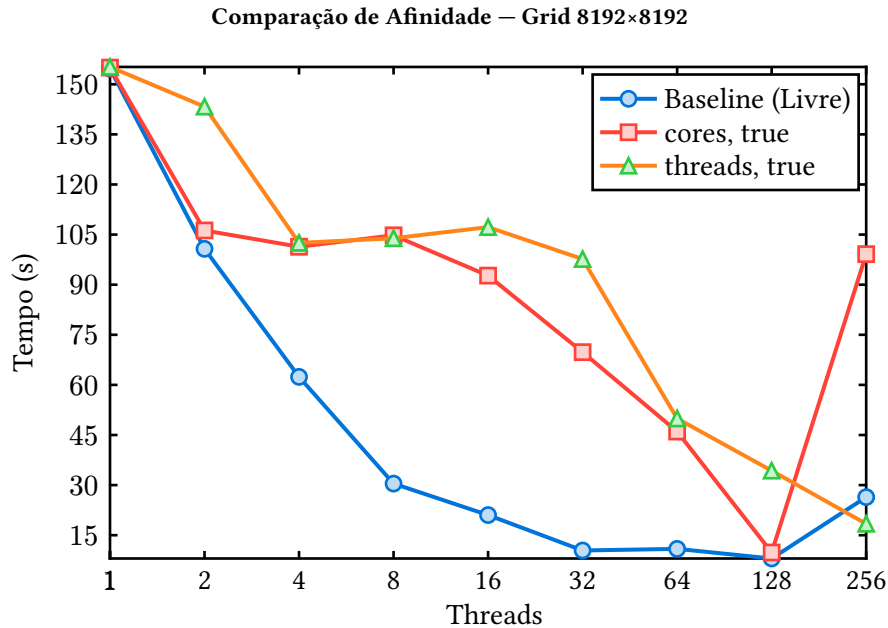


OMP_PLACES=threads, OMP_PROC_BIND=true



III. II. Comparação entre Afinidades para o Grid 8192×8192

Para facilitar a comparação direta, o gráfico a seguir sobrepõe as três estratégias para o maior grid testado (8192×8192).



Observa-se que, para até 128 threads, o escalonador padrão do sistema operacional (Baseline) lida de forma extremamente eficiente com a distribuição de carga. A curva começa a apresentar diferenças notáveis no limite de 256 threads, onde a arquitetura SMT (Simultaneous Multi-Threading) passa a ter contenção severa.

Neste cenário extremo (256 threads), a configuração `OMP_PLACES=cores` com `OMP_PROC_BIND=true` tem uma grave degradação (99 s). Em contrapartida, `OMP_PLACES=threads` com `OMP_PROC_BIND=true` obteve o melhor tempo (18,4 s).

IV. Análise de Speedup

A tabela abaixo apresenta o speedup $S = \frac{T(1)}{T(p)}$ para o grid 8192×8192. O tempo de execução sequencial (1 thread) é virtualmente idêntico nas três situações (155 s).

Threads	Baseline (Livre)	cores, true	threads, true
1	1	1	1
2	1.54	1.46	1.08
4	2.48	1.53	1.51
8	5.08	1.48	1.49
16	7.35	1.67	1.45
32	14.83	2.22	1.59
64	14.19	3.37	3.11
128	19.27	15.76	4.52
256	5.86	1.56	8.43

V. Conclusão

A escolha da afinidade de threads demonstrou impacto crítico no desempenho do solver Navier-Stokes, especialmente em alta concorrência em uma arquitetura NUMA complexa como a do nó [amd-512](#).

As principais observações físicas e arquiteturais incluem:

- **A Eficiência do Escalonador do SO:** Para até 128 threads (usando 1/4 da capacidade lógica da máquina), o `OMP_PROC_BIND=false` foi muito competitivo (alcançando Speedup de 18×). Como o nó não estava saturado, o escalonador do Linux conseguiu distribuir as threads eficientemente sem grande penalidade de migração.
- **Gargalo de Memória e SMT:** O solver Navier-Stokes é uma aplicação “memory-bound”. Em 256 threads, permitir a migração livre de threads (`false`) prejudica a localidade de cache, pois threads re-escaloadas precisam buscar dados iterativamente da RAM, aumentando as requisições na memória principal (cache misses).
- **Contenção em Granularidade de Núcleo:** A configuração `PLACES=cores`, `BIND=true` falhou severamente com 256 threads. Ao fixar a granularidade em núcleos físicos sem espalhamento lógico, o SMT pode não ter sido balanceado corretamente, gerando gargalos no barramento.
- **Localidade Forçada:** A configuração `PLACES=threads`, `BIND=true` extraiu o desempenho ótimo em 256 threads (18,4 s). Ao fixar as threads nos núcleos lógicos (nível SMT), o OpenMP assegura que cada bloco do grid permaneça no mesmo Cache L1/L2 durante toda a execução.

Recomenda-se o uso do escalonador livre (`BIND=false`) para cargas médias e a fixação severa (`PLACES=threads`, `BIND=true`) para extrair o máximo do hardware em regimes de alta saturação paralela.

VI. Anexo

```

#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define NT 5000
#define DX 0.1f
#define DY 0.1f
#define DT 0.001f
#define VISC 0.1f

typedef struct {
    int nx;
    int ny;
    float **data;
} Grid2D;

Grid2D alocar_grid(int nx, int ny, float valor_base) {
    Grid2D grid = {
        .nx = nx,
        .ny = ny,
        .data = (float **)malloc(nx * sizeof(float *)),
    };

    for (int i = 0; i < nx; i++) {
        grid.data[i] = (float *)malloc(ny * sizeof(float));
    }

    for (int i = 0; i < grid.nx; i++) {
        for (int j = 0; j < grid.ny; j++) {
            grid.data[i][j] = valor_base;
        }
    }

    // Perturbação central
    int cx = grid.nx / 2, cy = grid.ny / 2, r = 4;
    for (int i = cx - r; i <= cx + r; i++) {
        for (int j = cy - r; j <= cy + r; j++) {
            grid.data[i][j] = 10.0f;
        }
    }

    return grid;
}

void calcular_proximo_passo(const Grid2D u_current, const Grid2D u_next,
                           const float nu, const float dt, const float dx,
                           const float dy) {
    const int nx = u_current.nx;
    const int ny = u_current.ny;
    const float idx2 = 1.0f / (dx * dx);
    const float idy2 = 1.0f / (dy * dy);

#pragma omp parallel for
    for (int i = 1; i < nx - 1; i++) {

```

```

    for (int j = 1; j < ny - 1; j++) {
        const float laplaciano =
            (u_current.data[i + 1][j] - 2.0f * u_current.data[i][j] +
             u_current.data[i - 1][j]) *
            idx2 +
            (u_current.data[i][j + 1] - 2.0f * u_current.data[i][j] +
             u_current.data[i][j - 1]) *
            idy2;

        u_next.data[i][j] = u_current.data[i][j] + dt * nu * laplaciano;
    }
}

void salvar_metricas_csv(const char *nome_arquivo, int nx, int ny, int passos,
                        double tempo, int threads) {
    FILE *teste_existencia = fopen(nome_arquivo, "r");
    int precisa_cabecalho = (teste_existencia == NULL);
    if (teste_existencia != NULL) {
        fclose(teste_existencia);
    }

    FILE *f = fopen(nome_arquivo, "a");
    if (f == NULL) {
        fprintf(stderr, "Erro ao abrir o arquivo CSV para escrita.\n");
        return;
    }

    if (precisa_cabecalho) {
        fprintf(f, "tipo_execucao,nx,ny,passos_tempo,tempo_segundos,threads\n");
    }

    fprintf(f, "%d,%d,%d,%f,%d\n", nx, ny, passos, tempo, threads);

    fclose(f);
}

void liberar_grid(Grid2D *grid) {
    for (int i = 0; i < grid->nx; i++) {
        free(grid->data[i]);
    }
    free(grid->data);
    grid->data = NULL;
}

int main(void) {
    omp_set_dynamic(0);
    const char *nome_csv = "resultados.csv";

    int resolucoes[] = {1024, 2048, 4096, 8192};
    int total_execucoes = sizeof(resolucoes) / sizeof(resolucoes[0]);

    for (int threads = 1; threads <= 256; threads *= 2) {
        omp_set_num_threads(threads);
        for (int i = 0; i < total_execucoes; i++) {

```

```

int nx_atual = resolucoes[i];
int ny_atual = resolucoes[i];

printf("=====\n");
printf("Iniciando simulação para o grid: %dx%d\n", nx_atual, ny_atual);

Grid2D u_velA = alocar_grid(nx_atual, ny_atual, 1.0f);
Grid2D u_velB = alocar_grid(nx_atual, ny_atual, 1.0f);

Grid2D u_atual = u_velA;
Grid2D u_proximo = u_velB;

printf("Velocidade inicial no centro: %.2f\n",
       u_atual.data[nx_atual / 2][ny_atual / 2]);

double tempo_inicio = omp_get_wtime();

for (int t = 0; t < NT; t++) {
    calcular_proximo_passo(u_atual, u_proximo, VISC, DT, DX, DY);

    Grid2D temp = u_atual;
    u_atual = u_proximo;
    u_proximo = temp;
}

double tempo_fim = omp_get_wtime();
double tempo_total = tempo_fim - tempo_inicio;

printf("Velocidade final no centro após %d passos: %f\n", NT,
       u_atual.data[nx_atual / 2][ny_atual / 2]);
printf("Tempo total gasto: %f segundos\n", tempo_total);

salvar_metricas_csv(nome_csv, nx_atual, ny_atual, NT, tempo_total,
                   threads);

liberar_grid(&u_velA);
liberar_grid(&u_velB);
}
}

printf("=====\n");
printf("Todas as execuções finalizadas.\n");

return 0;
}

```